COMP 6713 Group Project

Emoji Intelligence Unit: SACARSM/IRONY Detection in social media text

To Sang Ngan (z5342080), Samyak Diwan (z5611048), Gudiraj Bal(z5386590), Junwei (z5328887)

Project Scope:

PART A: Problem Definition **Minimum: 10 credits**			**Fill your project details here**
	Points	**Details**	
NLP Problem	5 credits per problem	Examples: Question-answering, Sentiment Analysis, etc. (Refer to course modules)	**Sarcasm/Irony Detection in social media text** (binary text classification) Given a short social media message with emojis, predict whether it is sarcastic (1) or not sarcastic (0), along with the incorporation of both emojis and text.
Text Source/Domain	5 credits per text source/domains	Examples: News Articles, Medical papers, social media posts, etc.	Short, informal social media and chat-like messages containing emojis. Sources: Twitter-style posts (TweetEval) and Reddit comments (SARC 2.0)
PART B: Dataset Selection **Minimum: 20 credits**			
Use two existing datasets	10 credits	Publicly available datasets.	**Dataset 1:** TweetEval – Irony Subset Source: https://huggingface.co/datasets/cardiffnlp/tweet_eval Labels: irony / non_irony (binary) **Dataset 2:** SARC 2.0 – Balanced Split Source: https://nlp.cs.princeton.edu/old/SARC/2.0/main/ Files: train-balanced.csv.bz2 + test-balanced.csv.bz2 Labels: sarcastic / not_sarcastic (binary) **Dataset 2 changed:** iSarcasm Source: https://github.com/iabufarha/iSarcasmEval Files: train_A_En and test_A _En

			Labels: sarcastic / not_sarcastic
Create your own labelled dataset	20 credits	Correct selection of labels, inter-annotator agreement	
Use an existing lexicon	10 credits	Examples: WordNet, medical ontology	Emoji Sentiment Ranking (ESR) v1.0 Source: https://www.clarin.si/repository/xmlui/handle/11356/1048 File: Emoji_Sentiment_Data_v1.0.csv
PART C: Modelling			
Implement a rule-based or statistical model as a baseline	-	Essential. **Please keep this approach simple – this is only a baseline.**	Logistic Regression with TF-IDF features (scikit-learn)
Minimum: 30 credits			
Use an existing pre-trained/ fine-tuned model	5 credits per model	You must compare the performance of multiple models to accrue credits, if you are only using available fine-tuned models or only prompting.	**Model 1**: cardiffnlp/twitter-roberta-base-irony Source: https://huggingface.co/cardiffnlp/twitter-roberta-base-irony **Model 2**: google-bert/bert-base-uncased Source: https://huggingface.co/google-bert/bert-base-uncased
Fine-tune a model based on a dataset	20 credits per out-of-the-box fine-tuning method	Examples: fine-tuned BERT, prefix-tuned LLaMA	**Fine-tune:** cardiffnlp/twitter-roberta-base Source: https://huggingface.co/cardiffnlp/twitter-roberta-base (Base version is diff from the pre-trained irony model above, we will fine tune this on our own)
Extend a method	30 credits per extended fine-tuning method	Examples of extension: modification of the loss function, incorporation of structured ontology, prompting method other than	

		zero/few-shot prompting, etc.	
Integrate a language model with external tools	20 credits	Usage of a library like LangChain/LangSmith	
PART D: Evaluation **Minimum: 20 credits**			
Quantitative Evaluation	10 credits	Appropriate metrics.	Macro F1, Accuracy, Precision, Recall, Confusion Matrix etc
Qualitative Evaluation	5 credits	Examine mis-classified instances and produce common error types	We will Inspect 10 misclassified items from the best-performing model. For each item we will record: text, true label, predicted label, probability, emoji list.
Command line testing	5 credits	Interface to test out the system. This can be executed using an input argument or input file.	Our script.. like predict.py example/ Argument: model emoji Output: Label = Sarcastic, prob = 0.85 emojis =
Demo	10 credits	A simple demo through Gradio (or equivalent).	

Report

Part A: Problem Definition

NLP Problem

The primary focus of this project is sarcasm and irony detection within social media text, which is a binary text classification task. The objective is to develop a model that can take a short informal social media text and predict whether it is sarcastic/ironic (1) or non-sarcastic/non-ironic (0). The core aspect of this problem is that sarcasm is often expressed in an indirect manner, where the actual meaning of the speaker is different from the literal meaning of the text. This creates a challenge for NLP since social media texts are often short-form and the context is limited. Emojis are particularly prevalent in this domain and can serve as paralinguistic cues that clarify, contradict, or amplify the sentiment of the accompanying text — for example, 😒 following an apparently positive statement strongly signals sarcastic intent. This project, therefore, treats emoji handling as an explicit experimental variable, investigating whether retaining, removing, or converting emojis into text aliases affects a model's ability to detect sarcasm, and whether emoji-derived sentiment features from the Emoji Sentiment Ranking lexicon provide additional signal beyond text alone.

Domain

The text domain for this project is short, informal social media messages containing emojis. Our main text sources are TweetEval and iSarcasm (English), both of which contain short-form online posts from Twitter, suitable for sarcasm detection. iSarcasm (English) is especially relevant because it targets intended sarcasm, where labels are provided by the authors of the texts themselves rather than inferred only through hashtags or third-party annotation, making it well-suited for studying more realistic sarcastic language. By combining textual content with emoji information, this project aims to improve the detection of subtle, context-dependent sarcasm in real-world online communication.

Part B: Dataset Selection

Existing Datasets:

Dataset 1 – TweetEval Irony (English)

TweetEval is a standardised benchmark for tweet classification hosted on HuggingFace, which consists of 7 heterogeneous tasks in Twitter [1], and its subset irony will be used in this assignment. This dataset is one of the most widely used benchmarks for tweet classification. By using this dataset for training and for comparing models, we are able to evaluate models' performance consistently and interpretably. Additionally, the two selected pretrained models, ***cardiffnlp/twitter-roberta-base-irony*** and ***pysentimiento/bertweet-irony***, are both trained on Twitter data, which can reflect the intended result if using data from social media.

This dataset contains 2,862 rows in training data, 955 rows in validation, and 784 rows in the testing set, which are tweets drawn from Twitter with binary labels (irony and non-irony). The class distribution in the training set is balanced, which contains 1445 irony tweets and 1417 non-irony tweets. Nevertheless, the testing set is slightly imbalanced, which consists of 311 ironic cases and 473 non-ironic cases (60% vs. 40%). This motivates our choice of using Macro-F1 as the primary evaluation metric rather than accuracy, as Macro-F1 assigns equal weight to each class and is more suitable for imbalanced datasets.

Dataset 2 – iSarcasm (English)

From the group project scope, SARC 2.0 was the second dataset for sarcasm detection, but we switched to iSarcasm as a result of its difficulty. During initial evaluation, we found that both pre-trained models scored approximately 0.50 Macro-F1 on the SARC, which is hard to tell if the models are randomly guessing or not. SARC is a dataset drawn from comments in Reddit posts, where sarcasm labels are inferred from the */s* tag added by the post author [2]. However, each instance in the dataset contains only a single comment without any contextual information from the parent thread. Sarcastic detection based on isolated text without access to its context is a well-known challenge in the field [3], as many samples are genuinely ambiguous. Therefore, we replaced SARC 2.0 with the iSarcasm dataset as our second evaluation target.

The iSarcasm dataset addresses the issue of label ambiguity by relying on annotations directly provided by the authors of the tweets (sarcastic or non-sarcastic). This approach enhances the reliability of the labels and reflects the authors' true intention [4]. The test set contains 1,400 rows with a significantly imbalanced class distribution. It consists of 200 sarcastic tweets and 1,200 non-sarcastic tweets (14% vs 86%), which will potentially lead to biased performance metrics and may not accurately reflect the actual model's ability to detect sarcasm.

Existing Lexicon:

External Lexicon - Emoji Sentiment Ranking (ESR)

As the external lexicon component of the project, Emoji Sentiment Ranking (ESR) v1.0 serves as an important basis to determine the emotions contained in emojis. ESR is a lexicon containing 969 Unicode emoji entries and their assigned sentiment on a scale from -1 to +1. This sentiment score is computed through 70,000 tweets that are labeled by 83 human annotators in the class of positive, negative, and neutral, and is defined as the normalized difference between positive and negative labels divided by the total number of annotations [5].

$$Sentiment\ Score = \frac{pos - neg}{pos + neg + neu}$$

In our baseline model, ESR is used as a feature rather than a standalone model. For each input text, we detect emojis and retrieve their ESR sentiment scores, then derive several emoji-level features: emoji count, unique emoji count, sentiment score sum, sentiment score mean, number of positive emojis, and number of negative emojis. These structured ESR features are combined with the TF-IDF representation of the text and fed into a logistic regression classifier. This enables the baseline to ingest both textual cues and emoji-based sentiment signals during prediction. We also compare this enriched baseline against a text-only TF-IDF baseline to determine whether ESR-based emoji information improves sarcasm detection performance.

Part C: Modelling & Experimental Setup

Statistical model/Rule-based model

Our baseline approach is a statistical text classification model based on TF-IDF features and Logistic Regression. After standardizing the datasets (Stage 1), each post is first preprocessed by normalizing user mentions and URLs and removing unnecessary whitespace. For the text-only baseline, we use a Mode T representation, where emojis are removed from the text so that the textual channel can be modeled separately from the emoji channel. The cleaned text is then transformed into a sparse TF-IDF representation using unigrams and bigrams, with lowercasing enabled and sublinear term-frequency scaling. This allows the model to capture important lexical patterns and short phrase-level cues that may signal sarcasm. A Logistic Regression classifier is then trained on these TF-IDF features as the main baseline model.

To extend this baseline, we incorporate ESR features alongside the TF-IDF text representation. For each post, emojis are identified and mapped to ESR sentiment scores, from which we derive six numeric features: emoji count, unique emoji count, sum of ESR sentiment scores, mean ESR sentiment score, positive emoji count, and negative emoji count. These ESR features are standardized using StandardScaler and concatenated with the sparse TF-IDF matrix to form a combined feature representation. A second Logistic Regression model is then trained on this combined input. This gives us two directly comparable baseline variants: (1) TF-IDF text only and (2) TF-IDF + ESR features. The purpose of this design is to test whether emoji-based sentiment information improves sarcasm detection beyond text alone.

Existing pre-trained models

RoBERTa

RoBERTa [6] is a robustly optimised extension of the BERT transformer, which removes the Next Sentence Prediction objective, is trained on data for an extended period of time, and makes use of dynamic masking. Cardiff NLP pre-trained a RoBERTa-base model on ~58 million tweets, which were then fine-tuned on the TweetEval irony task, making it the baseline for Twitter irony detection. The reason for choosing RoBERTa was that its irony-specific fine-tuning on TweetEval means the model has learnt, to an extent, to associate Twitter-specific sarcasm indicators, such as hashtags or exaggerated praise or slang, with the irony label, giving it a headstart over a regular classifier in sarcasm detection.

BERTweet

BERTweet [8] is the first large-scale LM pre-trained only on English tweets using a ~850 million tweet corpus and using the RoBERTa training method. BERTweet adds an irony-specific fine-tuning layer. Sarcasm on Twitter is seen to be expressed through conventions that are not commonly found in general-domain text - the usage of emojis, abbreviations, misspellings, and a 280-character compression. Since BERTweet is trained on a substantially larger corpus, it is believed to have internalised these conventions at the subword level, which gives its context representations a better understanding of social media cues.

Why these two models?

RoBERTa is task-specialised, where it was pre-trained on a general web text and tweets, which were then fine-tuned on irony. BERTweet is domain-specialised, where it was pre-trained deeper on tweet-specific language, then fine-tuned for irony. This allows us to also understand what plays a larger role in sarcasm detection: Task-specific fine-tuning on irony labels or Domain-specific pre-training on tweet language.

Experimental setup

We made use of several key Libraries and tools. For the DeepLearning framework, we made use of **PyTorch**. For Transformers and Model Hub, we used **Hugging Face**. For Data Handling, we made use of **pandas** and **numpy**. For Evaluation and metrics, we made use of **scikit-learn** and **matplotlib.** For Emoji Handling, we made use of **emosent-py**.

Our overall experimental setup is split into 5 stages:

- **Stage 1:** Data collection, preprocessing, and enrichment
- **Stage 2:** Pre-trained inference-only baseline
- **Stage 3:** Fine-tuning stage
- **Stage 4:** Evaluation stage
- **Stage 5:** Ready for CLI

Stage 1: Data collection, preprocessing, and enrichment

Here, the datasets were collected to ensure evaluation across the different domains and annotation styles. TweetEval was downloaded via HuggingFace, with its 2,862 training, 955 validation, and 784 test examples. iSarcasm was downloaded and then uploaded locally, where task_A_En_test was read and normalized into text/label, which produced 1,400 test examples with 1,200 non-sarcastic and 200 sarcastic. These datasets were then standardised to a common schema with columns being text, label, dataset, and split. They were then

concatenated into train_unified.csv, val_unified.csv, and test_unified.csv, which are then saved to the data folder.

Next, **Emoji Sentiment Enrichment** is carried out, where each text was passed through the Emoji Sentiment Ranking (ESR) lexicon using the emosent-py library that produces scores for 969 emojis. Six tweet features were computed and inserted into each row.

- Emoji_count - total number of emojis that appear in text
- Emoji_unique_count - Number of distinct emojis
- ESR_sent_sum - Sum of ESR scores across emojis in the text
- ESR_sent_mean - Mean ESR score
- Pos_emoji_count - Number of emojis with ESR score > 0
- Neg_emoji_count - Number of emojis with ESR score < 0

Each enriched dataframe is then stored in train_enriched.csv, test_enriched.csv, and val_enriched.csv.

From here, we look into the effects of emoji representation on model performance. Here, **Emoji Model Preprocessing** was conducted, where three text variations were generated for each example.

- Mode T (Strip) - All emojis are removed, leaving only text
- Mode K (Keep) - Emojis are left in place as raw Unicode
- Mode D (Demojize) - Emoji is replaced by its Unicode name in colons, making its sentiment readable as subwords within colons

Before each transformation was applied, the texts were normalised by stripping URLs, @User mentions, and whitespace. The three-mode columns are then stored by the original text on the train, test, and validation dataframes.

Stage 2: Pre-trained inference-only baseline

In stage 2, we run both pre-trained models in inference mode - here, no training is performed. This is to measure the model's performance purely to its pre-training and irony-specific fine-tuning, where Macro-F1 values are computed and measured against in the later Stages 3 and 4. Models were both loaded through the Hugging Face AutoTokenizer and AutoModelForSequenceClassification APIs. A reusable inference pipeline is implemented, and raw texts are wrapped in a minimal TextDataset, which is then fed through the DataLoader. Here, each batch is then sent through the model, where an evaluation function then carries out the prediction, with the computed Macro-F1 using sklearn prints a full classification_report with precision, recall, and F1 computations in a labelled confusion matrix. Macro-F1 is chosen as it weighs both classes equally, regardless of the imbalance between them, for example, iSarcasm

6:1 non-sarcastic to sarcastic ratio. Here, accuracy would be misleading as the majority-class predictor. Results are then saved in stage2_baseline.csv, which is then discussed during **Evaluation**. Here, the results are also shown within the notebook, in a classification matrix that displays the Macro-F1, Precision, and Recall.

Stage 3: Fine-tuning stage

In this stage, the RoBERTa model is adapted for binary sarcasm/irony detection using a standard sequence classification head. The head weights are freshly initialised since the base checkpoints are trained on masked language modelling, while the LM head weights are intentionally dropped. We use the RoBERTa base model with a max sequence length of 128 tokens and a batch size of 32. We trained it for 5 epochs at a learning rate of 1e-5 (AdamW), with a weight decay of 0.01. Early stopping is also used, validating Macro-F1 to save the best checkpoint when the model improves. This stage is then run on a T4 GPU via Collab.
To directly address the research question on emoji handling, we trained three separate model instances under different emoji representation strategies — Mode T (strip), Mode K (keep raw emojis), and Mode D (demojize), with all other hyperparameters held constant. This three-way ablation isolates the effect of emoji representation on model performance.

Stage 4: Evaluation stage

The primary metrics computed for every model compute the Macro-F1, which is the primary optimisation target. It takes the average F1 scores across both classes without weighing them by class size, making it robust to class imbalance. Every evaluation also comes with a confusion matrix below:

	Predicted: Not Sarcastic	**Predicted: Sarcastic**
Actual: Not Sarcastic	True Negatives (TN)	False Positives (FP)
Actual: Sarcastic	False Negatives (FN)	True Positives (TP)

This lets us measure how the model fails where it tends to miss sarcasm with a high FN, or falsely flag normal text with a high FP.

Stage 5: Ready for CLI

To satisfy the command-line testing requirement, we have implemented predict.py, a standalone inference script that allows users to test the sarcasm detection system on any arbitrary input text without requiring access to the training notebook or dataset files.

Usage:

The script accepts two input modes:

Single Text Input	python predict.py --model roberta_K --emoji_mode K --text "Great job cancelling again 😒"
File Input	python predict.py --model roberta_K --emoji_mode K --input_file MISC/test_inputs.txt

Example Output:

```
Text : I like the course NLP❤️
Label : not_irony
Prob(irony) : 0.198
Prob(not_irony): 0.802
Emojis : ['❤️']
ESR sentiment : 0.746
Emoji count : 1
Processed text: I like the course NLP❤️
```

```
Text : Oh wow! Shocking. Great
Label : irony
Prob(irony) : 0.9402
Prob(not_irony): 0.0598
Emojis : []
ESR sentiment : 0.0
Emoji count : 0
Processed text: Oh wow! Shocking. Great
```

Model selection & Threshold:

The default chosen model is **RoBERTa_k** (keep emojis mode) as it achieved the highest

TweetEval Macro-F1 of 0.75+ across all 3 fine-tuned variants. The default classification threshold is set to 0.7 rather than the standard 0.5.

Part D: Evaluation

Quantitative Evaluation

Stage 1 Evaluation: TF-IDF

Final Stage 1 results:

variant	dataset	n	f1_sarcastic	precision_sarcastic	recall_sarcastic	macro_f1	pr_auc	roc_auc	balanced_accuracy	accuracy	tn	fp	fn	tp
TFIDF_text_only	overall	29680	0.481290	0.485870	0.476796	0.505610	0.485032	0.506707	0.505645	0.506806	8251	7186	7452	6791
TFIDF_text_only	isarcasm	1400	0.243655	0.163265	0.480000	0.473716	0.192099	0.574479	0.535000	0.574286	708	492	104	96
TFIDF_text_only	sarc	27496	0.487383	0.499198	0.476114	0.499524	0.496339	0.496652	0.499791	0.499818	7205	6559	7194	6538
TFIDF_text_only	tweeteval	784	0.520730	0.537671	0.504823	0.610624	0.592804	0.656601	0.609705	0.631378	338	135	154	157
TFIDF_plus_ESR	overall	29680	0.483182	0.488415	0.478059	0.507982	0.487244	0.510993	0.508026	0.509232	8305	7132	7434	6809
TFIDF_plus_ESR	isarcasm	1400	0.279006	0.192748	0.505000	0.513780	0.220042	0.625125	0.576250	0.627143	777	423	99	101
TFIDF_plus_ESR	sarc	27496	0.487736	0.498820	0.477134	0.499192	0.496257	0.496548	0.499429	0.499454	7181	6583	7180	6552
TFIDF_plus_ESR	tweeteval	784	0.526138	0.553191	0.501608	0.618967	0.597197	0.667356	0.617611	0.641582	347	126	155	156

Figure 3: Baseline TF-IDF model, full results when running TF-IDF on text and ESR

Overall Performance:
Both TF-IDF versions perform only slightly better than chance across all the datasets combined. **TF-IDF_text_only** achieves an overall Macro-F1 of **0.506,** and **TFIDF_plus_ESR** achieves **0.50,8** with just a difference of **0.002.** This shows near-random aggregate performance for TF-IDF when trying to detect sarcasm. Here, the bag-of-words representation only captures lexical frequency but shows no real ability for modelling the contextual incongruity that helps identify sarcasm. Individual breakdowns and confusion matrices can be found in **Additional Resources**.

Per-Dataset Breakdown

<u>TweetEval</u>
TF-IDF performs best on the TweetEval domain with a substantial gap over the other datasets.

Variant	Macro-F1	ROC-AUC	Precision (sarc)	Recall (sarc)
TFIDF_text_only	**0.611**	0.657	0.538	0.505
TFIDF_plus_ESR	**0.619**	0.667	0.553	0.502

With a Macro-F1 of **0.611-0.619**, it shows that it does slightly better than chance. This is suspected to be the case due to Twitter irony co-occurring with certain markers like hashtags,

adjacent terms, or exaggerated positive language and phrases. This is then exploited by TF-IDF's term frequency weighting. The ROC-AUC - **Receiver Operating Characteristic Area Under the Curve,** measures how well a binary classifier separates the two classes across all possible decision thresholds. It measures the TP-rate against the FR-rate. In this instance, it scores a 0.657-0.667, which shows that TF-IDF is genuinely able to discriminate between sarcastic and non-sarcastic posts. The improvement in precision from **0.538** to **0.553** also indicates that ESR features strongly assist in reducing false positives on the dataset.

<u>SARC 2.0</u>

Here is where TF-IDF struggles alongside our other models, as evaluated later on.

Variant	Macro-F1	ROC-AUC	Balanced Accuracy
TFIDF_text_only	0.500	0.497	0.500
TFIDF_plus_ESR	0.499	0.497	0.499

At this stage, all metrics sit around ~0.5, suggesting it's no better than a random classifier. This is likely because Reddit sarcasm is expressed through propositional irony and subtext rather than any surface-level patterns.

<u>iSarcasm</u>

This dataset reveals an issue with handling the class imbalance problem, given that iSarcasm has 1,200 non-sarcastic vs 200 sarcastic texts.

Variant	Macro-F1	F1 (sarc)	Precision (sarc)	Recall (sarc)
TFIDF_text_only	0.474	0.244	0.163	0.480
TFIDF_plus_ESR	0.514	0.279	0.193	0.505

The classifier is able to achieve moderate recall on the sarcastic class with values 0.48-0.51, but with bad precision values 0.153-0.19,3 suggesting that every 4 in 5 sarcastic predictions is a FP. This indicates that the model is over-predicting sarcasm on a dataset where it is rare. This happens mostly when there is a class imbalance where precision-recall is traded off, in addition to TD-IDF's inability to capture context. Notable to mention is how, ESR outperforms text_only, with a +0.40 Macro-F1 improvement and an ROC-AUC improvement of +0.05, as indicated in

Figure 1. Here is where ESR is seen to have contributed the most in the results, likely due to Twitter posts with iSarcasm containing more telling emojis of sarcastic intent.

Stage 2 Evaluation:

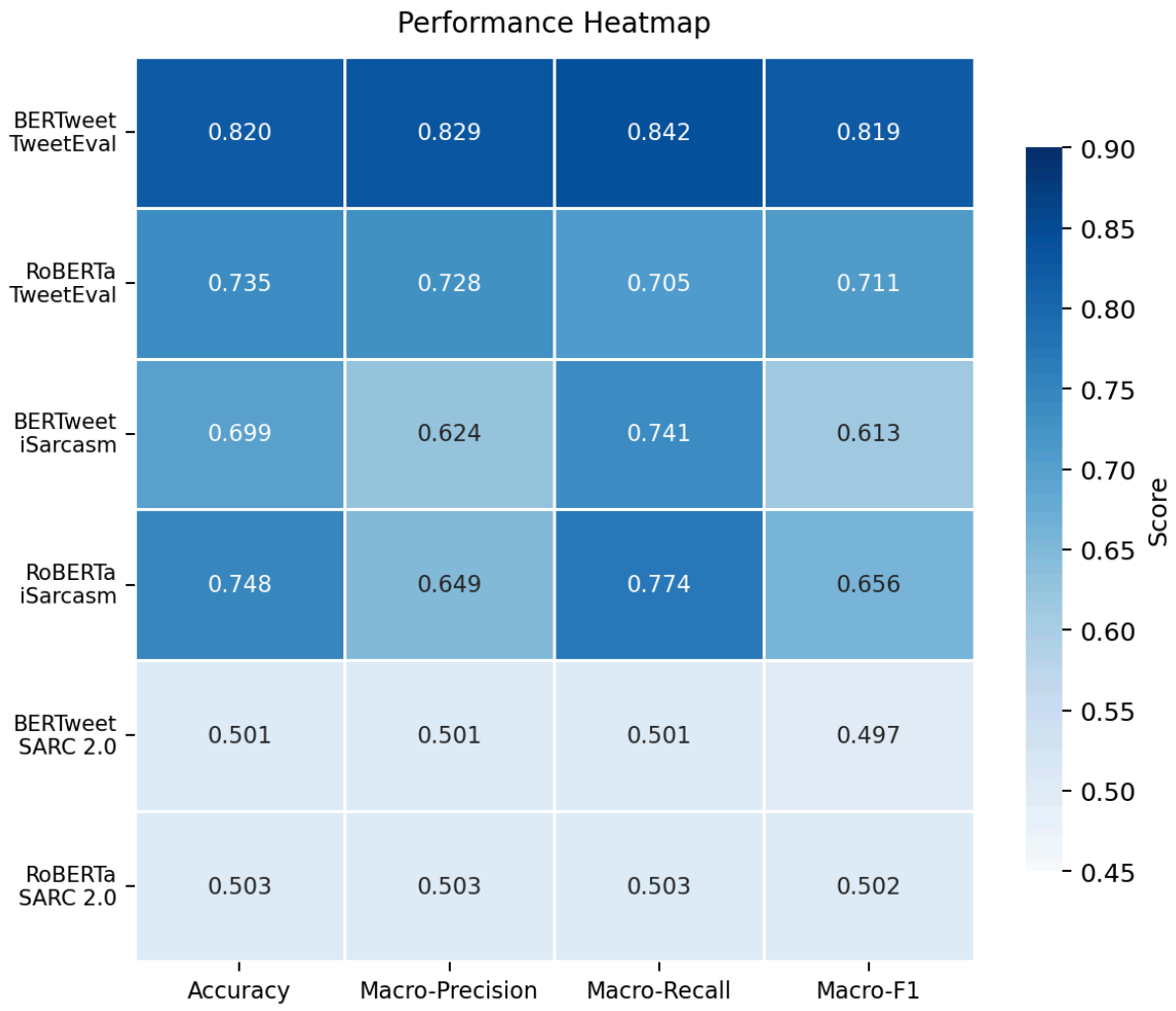


Figure 4: Performance Heatmap of baseline models on inference mode.

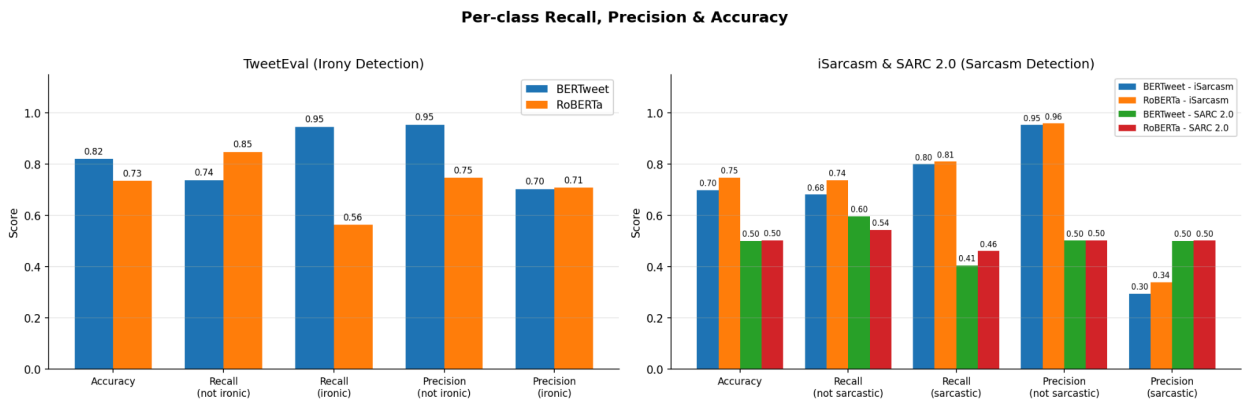


Figure 5: Per-class Recall, Precision, and Accuracy

Stage 2 Analysis

<u>TweetEval</u>

The performance of both models on TweetEval is significantly better than on the other two datasets. This is expected, as both models' irony detection ability was fine-tuned on the Twitter corpus, which makes TweetEval an in-distribution evaluation. The result shows that BERTweet achieved a Macro-F1 of 0.8193, while RoBERTa achieved a Macro-F1 of 0.7106. It is worth noting that BERTweet has a very high recall of 0.95 for the ironic class, but a lower precision of 0.70 on the same class, which may indicate that BERTweet can identify most sarcastic tweets, but at the cost of more false positives. RoBERTa shows a more conservative pattern, with lower recall (0.56) and higher precision (0.71) on the ironic class. Since it is less likely to be predicted as ironic, it leads to missing more true positives.

<u>iSarcasm</u>

Sarcasm detection is much harder than irony detection as sarcasm heavily relies on author intent and contextual information rather than surface- level linguistic markers. This is also presented in the result table, that the Macro F1 score of both models significantly dropped when shifted to sarcasm detection. The Macro-F1 score of BERTweet falls sharply from 0.819 to 0.613, whereas RoBERTa declines marginally from 0.711 to 0.656, which outperforms the BERTweet model.

Based on the per-class result in Figure 2, both models still exhibit a strong bias on prediction. The Recall on sarcastic class is relatively high, reaching 0.80 for BERTweet and 0.81 for RoBERTa, yet Precision is extremely low (BERTweet 0.30, RoBERTa 0.34). This result might potentially be caused by the extremely imbalanced dataset that iSarcasm has, in which the number of testing cases for the non-sarcastic class is 6 times more than that for the sarcastic. This bias generates a

high false positive rate, with many non-sarcastic tweets incorrectly identified as sarcastic, which substantially drives down Macro-F1 for both models. One reason RoBERTa might outperform BERTweet is that its base model was trained on a more extensive corpus rather than focusing specifically on Twitter, which provides better generalization ability [6]. This is reflected in the recall results that under the high precision rate for both models, RoBERTa achieved a recall of 0.74 in the non-sarcastic class, which is 0.06 higher than BERTweet.

SARC 2.0

Both models score approximately 0.50 on Macro-F1 and Accuracy on SARC 2.0, which is equivalent to random guessing. This result aligns with our earlier reasoning in data exploration that SARC annotations are grounded in full Reddit comment threads without access to contextual information, which results in poor performance from both models. This finding also validates our decision to switch to iSarcasm as the dataset for Sarcasm detection.

Stage 3 Evaluation:

Model	Stage	TweetEval F1	iSarcasm F1
RoBERTa	Pre-trained	0.711	0.656
BERTweet	Pre-trained	0.819	0.613
RoBERTa fine-tuned (T)	Fine-tuned	0.719	0.711
RoBERTa fine-tuned (D)	Fine-tuned	0.724	0.722
RoBERTa fine-tuned (K)	**Fine-tuned**	**0.752**	**0.723**

TweetEval

All three fine-tuned RoBERTa variants improved over the pretrained RoBERTa in Stage 2 (0.711), with the best performing being Mode K at (0.752) on TweetEval, with a +0.041 increase.

Run	Test Macro-F1	Δ vs RoBERTa pre-trained	Δ vs TF-IDF best
Fine-tuned (T)	0.719	+0.008	+0.100
Fine-tuned (D)	0.724	+0.013	+0.105

Fine-tuned (K)	**0.752**	**+0.041**	**+0.133**

Mode K's results on BERTweet show a reduction from 0.108 (pre-trained) to 0.067 just through the choice of emoji representation without architectural change. Early stopping was done on Mode D at Epoch 3, and a lower test score (0.724) suggests that the demojize texts introduce noise that causes overfitting on the small TweetEval training set. These results can be seen in the Additional Resource section.

<u>iSarcasm</u>

Run	**iSarcasm F1**	**Δ vs RoBERTa pre-trained (0.656)**	**Δ vs BERTweet pre-trained (0.613)**
Fine-tuned (T)	0.711	**+0.055**	+0.098
Fine-tuned (D)	0.722	**+0.066**	+0.109
Fine-tuned (K)	**0.723**	**+0.067**	**+0.110**

iSarcasm evaluation delivered the most interesting results within Stage 3. All three runs are seen to outperform both Stage 2 RoBERTa baselines (0.656) and Stage 2 BERTweet baselines (0.613) on iSarcasm. The +0.055 to +0.067 gain over the best Stage 2 models on iSarcasm indicates a substantial improvement, including on all three emoji modes (0.711/0.722/0.723) - despite the spread being 0.012 compared to 0.033 on TweetEval. This indicates that emoji mode selection plays a small role in iSarcasm as compared to TweetEval. iSarcasm tweets are labelled by authors where sarcasm is expressed through linguistic structure and context over cues such as emojis.

Mode	**TweetEval F1**	**iSarcasm F1**	**Average**
T (Strip)	0.719	0.711	0.715
D (Demojize)	0.724	0.722	0.723
K (Keep)	**0.752**	**0.723**	**0.738**

The ordering also shows how demojization may slightly be more informative for author-labelled sarcasm, where emoji names (:rolling_eyes:,:heart:) carry more signal that raw image. If a user were to choose an emoji mode across domains, Mode K should be the safest choice despite its advantage being concentrated on TweetEval, and since emojis tend to be helpful.

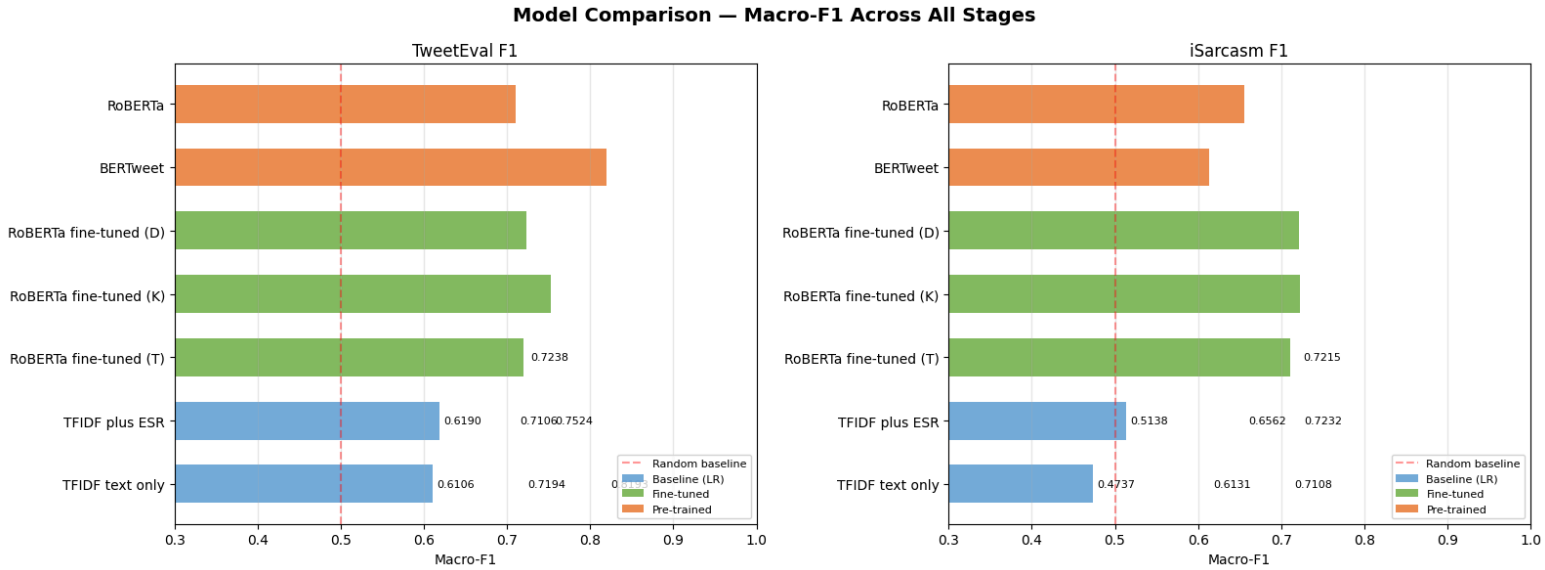


Figure 6: Model Comparison across stages.

<u>Overall Summary and Learning Outcomes for Stage TD_IDF + 2+3</u>

At each stage, there is a monotonic improvement from TF_IDF through pre-trained inference, followed by fine-tuning.

TweetEval: TF-IDF (0.619) -> RoBERTa pre-trained (0.711, +0.092) -> RoBERTa fine-tuned K (0.752, +0.041) -> BERTweet pre-trained (0.819)

iSarcasm: TF-IDF (0.514) -> BERTweet pre-trained (0.613, +0.099) -> RoBERTa pre-trained (0.656, +0.043) -> RoBERTa fine-tuned K (0.723, +0.067).

The jump from TF-IDF to transformer indicates how contextual pre-training is the most influential when detecting irony/sarcasm. Fine-tuning then adds a meaningful increase on datasets as indicated by the +0.041 on TweetEval and +0.067 on iSarcasm, demonstrating that task-specific adaptation and pre-training add to the model's improvement.

An interesting result demonstrated across Stage 2 and 3 shows that BERTweet, which is the stronger model on TweetEval (0.819), is weaker on iSarcasm(0.613), but the fine-tuned RoBERTa surpasses both. In Stage 2, iSarcasm confusion matrices demonstrate a precision of 0.295, while RoBERTa generates a precision of 0.340. BERTweet has learned a lower irony decision threshold from its large corpus of ~850 million tweets. Here, it detects irony-adjacent patterns like exaggeration, hyperbole, or emoji clusters that are common on Twitter but do not correspond to sarcasm that is **intended** as self-reported by authors. Here, BERTweet, which was optimised for tag-based irony, but overshoots on iSarcasms author-labelled sarcastic tweets. This also suggests that the datasets were likely built on different annotation strategies, where one is author-labelled (iSarcasm), and the other is guideline-abiding tweet annotation (TweetEval). Here, the fine-tuned RoBERTa performs better due to the additional fine-tuning on TweetEval's labelled examples, allowing it to make stronger decisions and learn patterns that guarantee ironic versus non-ironic. Therefore, the improvement is a calibration improvement as opposed to a representational one.

Qualitative Evaluation

For the Qualitative Error Analysis, we looked at 10 misclassified examples and split them into two distinct failure modes:

Error Type	Count	Direction	Core Cause
False Positive	5	not_irony -> predicted irony	Surface-level irony triggers without genuine irony
False Negative	5	irony -> predicted not_irony	Irony signalled through markers, the model ignores

Misclassified examples can be viewed in the **Additional Resources** section. Based on the analysis of the three examples, we can boil it down to three systematic weaknesses that we can generalise to the broader distribution.

1. <u>Lexical over-reliance</u>

The model can be seen to have learnt that specific tokens like “#irony, lol, at least, just love” are strong indications that the sentence may be ironic, but predicts them as such indiscriminately when appearing in non-ironic contexts. This is due to the training being tag-based distant supervision. This can be seen in Examples 2, 3, and 4. Here, Example 2 uses #irony in its literal sense, but the model treats it as deterministic evidence that it is ironic.

```
Text : @user @user A thief, a liar, a vote rigger, mocks those who use food banks and an online bully #irony
True : not_irony
Predicted : irony
Prob(irony): 0.8759999871253967
Emojis : []
ESR score : 0.0
```

2. <u>Underweighted linguistic markers</u>

For example, “#not” convention and obvious disclaimers such as “justajoke” and ESR-conflicting emoji pairs are genuine irony signals that are likely underweighted. The training distribution in TweetEval likely under-represents these layers compared to how often they are used daily. This is notable in examples 7, 8, and 9. Here in Example 7, irony is signalled three times, with the usage of conflicting emojis (“smile”, “disappointed”), the “#not” tag, and the classic “positive sentence [negative]” template. The model still assigns a 0.236 probability. Therefore, the ESR score of 0.135 obscured the “disappointed” emojis' negative signal, showcasing how averaging the ESR over conflicting emoji masks the overall sarcastic template of the sentence.

```
Text : I just love being ignored😊 |#not😢
True : irony
Predicted : not_irony
Prob(irony): 0.23600000143051147
Emojis : ['😊', '😢']
ESR score : 0.135
```

3. <u>Situational and World-Knowledge Irony</u>

Here, two clear examples, 6 and 10, require reasoning that goes beyond matching tokens, likely requiring general knowledge or context.

In Example 6, the sentence is extremely noisy with a lot of repetition, which is likely outside of the model's training distribution. Any preprocessing strips meaningful structure or coherent irony signals that cause the model to fail due to its need for grammatical coherence.

```
Text : Coin ! LeCoinCanard: Coin ! LeCoinCanard: Coin ! have a wank over this hashtag: Fazzanelli: JeffRoylat have a wank over this hashtag #Not...
True : irony
Predicted : not_irony
Prob(irony): 0.1420000046491623
Emojis : []
ESR score : 0.0
```

In Example 10, the irony is situational; it's a contradiction between the safety training context and the workers' unsafe behaviours. Here, the model is expected to infer that safety training should promote healthy behavior, which requires general/world knowledge. This is therefore likely treated as an observation with a rating of 0.521, indicating a lower confidence in resolving it.

```
Text : #Hazard worker training. 1st break there was 3 doz donuts and half class outside to smoke. #fail #nomealfitters #irony @user @use
True : irony
Predicted : not_irony
Prob(irony): 0.5210000276565552
Emojis : []
ESR score : 0.0
```

Additional Resources:

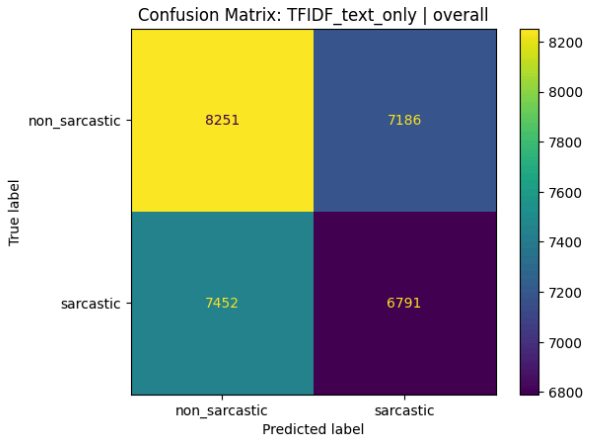
Stage 1-2 TF-IDF_text_only Results:

```
TFIDF_text_only | OVERALL
        variant dataset     n  accuracy  balanced_accuracy  precision_sarcastic  recall_sarcastic  f1_sarcastic  macro_f1  roc_auc   pr_auc   tn   fp   fn   tp
TFIDF_text_only overall 29680  0.506806           0.505645              0.48587          0.476796       0.48129   0.50561 0.506707 0.485032 8251 7186 7452 6791

Classification report:
               precision    recall  f1-score   support

non_sarcastic     0.5254    0.5345    0.5299     15437
    sarcastic     0.4859    0.4768    0.4813     14243

     accuracy                         0.5068     29680
    macro avg     0.5057    0.5056    0.5056     29680
 weighted avg     0.5065    0.5068    0.5066     29680
```

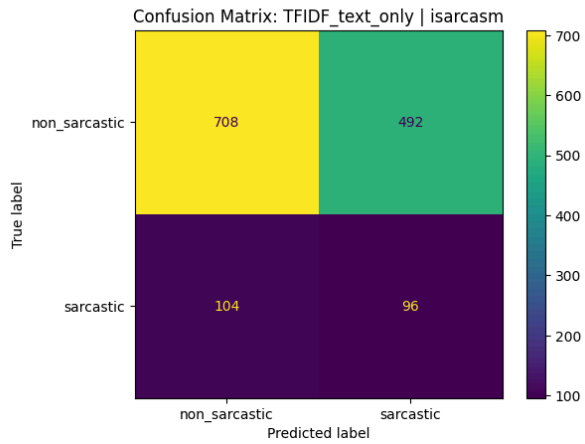


```
TFIDF_text_only | ISARCASM
        variant  dataset    n  accuracy  balanced_accuracy  precision_sarcastic  recall_sarcastic  f1_sarcastic  macro_f1  roc_auc   pr_auc  tn  fp  fn  tp
TFIDF_text_only isarcasm 1400  0.574286              0.535             0.163265              0.48      0.243655  0.473716 0.574479 0.192099 708 492 104  96

Classification report:
               precision    recall  f1-score   support

non_sarcastic     0.8719    0.5900    0.7038      1200
    sarcastic     0.1633    0.4800    0.2437       200

     accuracy                         0.5743      1400
    macro avg     0.5176    0.5350    0.4737      1400
 weighted avg     0.7707    0.5743    0.6380      1400
```

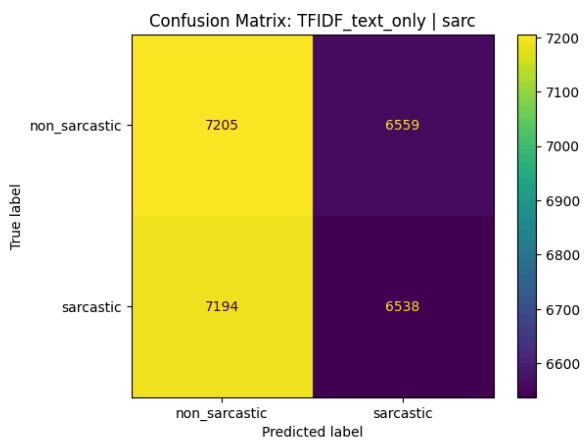



```
TFIDF_text_only | SARC
        variant dataset     n  accuracy  balanced_accuracy  precision_sarcastic  recall_sarcastic  f1_sarcastic  macro_f1  roc_auc   pr_auc   tn   fp   fn   tp
TFIDF_text_only    sarc 27496  0.499818           0.499791             0.499198          0.476114      0.487383  0.499524 0.496652 0.496339 7205 6559 7194 6538

Classification report:
               precision    recall  f1-score   support

non_sarcastic     0.5004    0.5235    0.5117     13764
    sarcastic     0.4992    0.4761    0.4874     13732

     accuracy                         0.4998     27496
    macro avg     0.4998    0.4998    0.4995     27496
 weighted avg     0.4998    0.4998    0.4995     27496
```

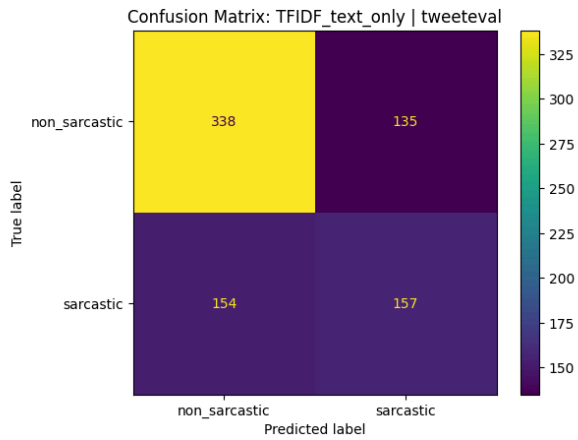


```
TFIDF_text_only | TWEETEVAL
        variant   dataset   n  accuracy  balanced_accuracy  precision_sarcastic  recall_sarcastic  f1_sarcastic  macro_f1  roc_auc   pr_auc  tn  fp  fn  tp
TFIDF_text_only tweeteval 784  0.631378           0.609705             0.537671          0.504823       0.52073  0.610624 0.656601 0.592804 338 135 154 157

Classification report:
               precision    recall  f1-score   support

non_sarcastic     0.6870    0.7146    0.7005       473
    sarcastic     0.5377    0.5048    0.5207       311

     accuracy                         0.6314       784
    macro avg     0.6123    0.6097    0.6106       784
 weighted avg     0.6278    0.6314    0.6292       784
```



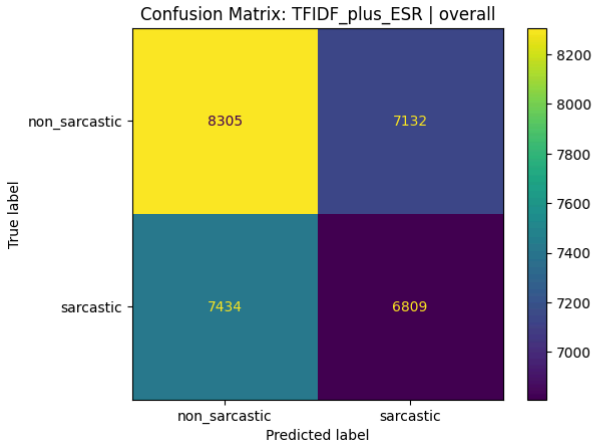
Stage 1-2 TF-IDF_ESR Results:

```
TFIDF_plus_ESR | OVERALL
       variant dataset     n  accuracy  balanced_accuracy  precision_sarcastic  recall_sarcastic  f1_sarcastic  macro_f1  roc_auc   pr_auc   tn   fp   fn   tp
TFIDF_plus_ESR overall 29680  0.509232           0.508026             0.488415          0.478059      0.483182  0.507982 0.510993 0.487244 8305 7132 7434 6809

Classification report:
               precision    recall  f1-score   support

non_sarcastic     0.5277    0.5380    0.5328     15437
    sarcastic     0.4884    0.4781    0.4832     14243

     accuracy                         0.5092     29680
    macro avg     0.5080    0.5080    0.5080     29680
 weighted avg     0.5088    0.5092    0.5090     29680
```

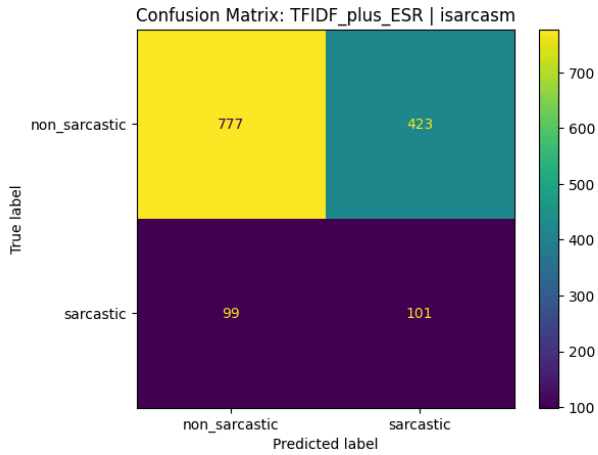


```
TFIDF_plus_ESR | ISARCASM
       variant  dataset    n  accuracy  balanced_accuracy  precision_sarcastic  recall_sarcastic  f1_sarcastic  macro_f1  roc_auc   pr_auc  tn  fp  fn  tp
TFIDF_plus_ESR isarcasm 1400  0.627143            0.57625             0.192748             0.505      0.279006   0.51378 0.625125 0.220042 777 423  99 101

Classification report:
               precision    recall  f1-score   support

non_sarcastic     0.8870    0.6475    0.7486      1200
    sarcastic     0.1927    0.5050    0.2790       200

     accuracy                         0.6271      1400
    macro avg     0.5399    0.5762    0.5138      1400
 weighted avg     0.7878    0.6271    0.6815      1400
```

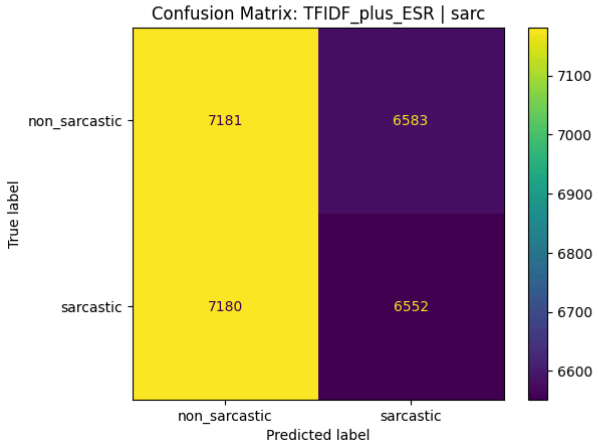



```
TFIDF_plus_ESR | SARC
       variant dataset     n  accuracy  balanced_accuracy  precision_sarcastic  recall_sarcastic  f1_sarcastic  macro_f1  roc_auc   pr_auc   tn   fp   fn   tp
TFIDF_plus_ESR    sarc 27496  0.499454           0.499429              0.49882          0.477134      0.487736  0.499192 0.496548 0.496257 7181 6583 7180 6552

Classification report:
               precision    recall  f1-score   support

non_sarcastic     0.5000    0.5217    0.5106     13764
    sarcastic     0.4988    0.4771    0.4877     13732

     accuracy                         0.4995     27496
    macro avg     0.4994    0.4994    0.4992     27496
 weighted avg     0.4994    0.4995    0.4992     27496
```

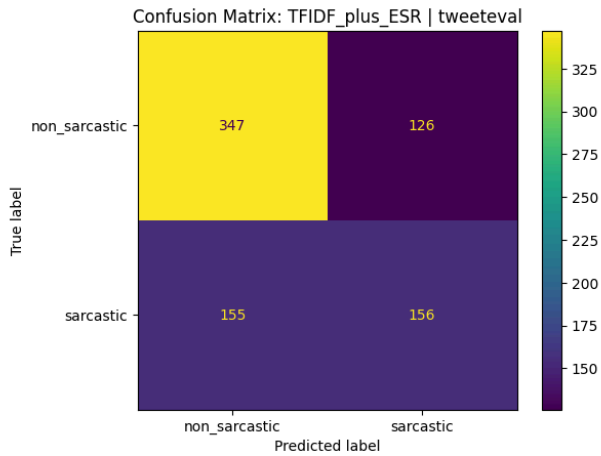


```
TFIDF_plus_ESR | TWEETEVAL
        variant   dataset   n  accuracy  balanced_accuracy  precision_sarcastic  recall_sarcastic  f1_sarcastic  macro_f1  roc_auc   pr_auc  tn  fp  fn  tp
TFIDF_plus_ESR tweeteval 784  0.641582           0.617611             0.553191          0.501608      0.526138  0.618967 0.667356 0.597197 347 126 155 156

Classification report:
               precision    recall  f1-score   support

non_sarcastic     0.6912    0.7336    0.7118       473
    sarcastic     0.5532    0.5016    0.5261       311

     accuracy                         0.6416       784
    macro avg     0.6222    0.6176    0.6190       784
 weighted avg     0.6365    0.6416    0.6381       784
```



Stage 3 Results and Confusion Matrix:

```
Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
 Epoch 1/5 | train_loss: 0.6855 | val_loss: 0.6520 | val_f1: 0.5863
 New best val_f1: 0.5863 — checkpoint saved
 Epoch 2/5 | train_loss: 0.5781 | val_loss: 0.5652 | val_f1: 0.7037
 New best val_f1: 0.7037 — checkpoint saved
 Epoch 3/5 | train_loss: 0.4734 | val_loss: 0.5764 | val_f1: 0.7142
 New best val_f1: 0.7142 — checkpoint saved
 Epoch 4/5 | train_loss: 0.4129 | val_loss: 0.5862 | val_f1: 0.7141
 No improvement. Patience: 1/2
 Epoch 5/5 | train_loss: 0.3659 | val_loss: 0.5937 | val_f1: 0.7141
 No improvement. Patience: 2/2
 Early stopping triggered.
Writing model shards: 100%|##########| 1/1 [01:07<00:00, 67.86s/it]
 Saved: models/stage3/roberta_tweeteval_D
 TweetEval Test Macro-F1: 0.7238
              precision    recall  f1-score   support

   not_irony       0.79      0.75      0.77       473
       irony       0.65      0.70      0.68       311

    accuracy                           0.73       784
   macro avg       0.72      0.73      0.72       784
weighted avg       0.74      0.73      0.73       784
```

```
Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
 Epoch 1/5 | train_loss: 0.6760 | val_loss: 0.6553 | val_f1: 0.5851
 New best val_f1: 0.5851 — checkpoint saved
 Epoch 2/5 | train_loss: 0.6002 | val_loss: 0.5987 | val_f1: 0.6387
 New best val_f1: 0.6387 — checkpoint saved
 Epoch 3/5 | train_loss: 0.5011 | val_loss: 0.5602 | val_f1: 0.7182
 New best val_f1: 0.7182 — checkpoint saved
 Epoch 4/5 | train_loss: 0.4300 | val_loss: 0.5861 | val_f1: 0.7225
 New best val_f1: 0.7225 — checkpoint saved
 Epoch 5/5 | train_loss: 0.3810 | val_loss: 0.5810 | val_f1: 0.7264
 New best val_f1: 0.7264 — checkpoint saved
Writing model shards: 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1/1 [00:58<00:00, 58.16s/it]
 Saved: models/stage3/roberta_tweeteval_K
 TweetEval Test Macro-F1: 0.7524
              precision    recall  f1-score   support

   not_irony       0.84      0.73      0.78       473
       irony       0.66      0.79      0.72       311

    accuracy                           0.76       784
   macro avg       0.75      0.76      0.75       784
weighted avg       0.77      0.76      0.76       784
```

```
Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
 Epoch 1/5 | train_loss: 0.6815 | val_loss: 0.6635 | val_f1: 0.5971
 New best val_f1: 0.5971 — checkpoint saved
 Epoch 2/5 | train_loss: 0.6168 | val_loss: 0.6197 | val_f1: 0.6366
 New best val_f1: 0.6366 — checkpoint saved
 Epoch 3/5 | train_loss: 0.5189 | val_loss: 0.6071 | val_f1: 0.6709
 New best val_f1: 0.6709 — checkpoint saved
 Epoch 4/5 | train_loss: 0.4555 | val_loss: 0.6128 | val_f1: 0.6861
 New best val_f1: 0.6861 — checkpoint saved
 Epoch 5/5 | train_loss: 0.4142 | val_loss: 0.6167 | val_f1: 0.6936
 New best val_f1: 0.6936 — checkpoint saved
Writing model shards: 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1/1 [01:05<00:00, 65.99s/it]
 Saved: models/stage3/roberta_tweeteval_T
 TweetEval Test Macro-F1: 0.7194
              precision    recall  f1-score   support

   not_irony       0.85      0.65      0.74       473
       irony       0.61      0.82      0.70       311

    accuracy                           0.72       784
   macro avg       0.73      0.74      0.72       784
weighted avg       0.75      0.72      0.72       784
```

```
Evaluating: roberta_tweeteval_T
Loading weights: 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 201/201 [00:00<00:00, 423.51it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]
  iSarcasm Macro-F1: 0.7108 (threshold=0.90)
              precision    recall  f1-score   support

not_sarcastic       0.91      0.94      0.93      1200
    sarcastic       0.56      0.45      0.50       200

     accuracy                           0.87      1400
    macro avg       0.74      0.69      0.71      1400
 weighted avg       0.86      0.87      0.86      1400
```

```
Evaluating: roberta_tweeteval_K
Loading weights: 100% ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 201/201 [00:00<00:00, 310.42it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]
  iSarcasm Macro-F1: 0.7232 (threshold=0.90)
              precision    recall  f1-score   support

not_sarcastic       0.91      0.95      0.93      1200
    sarcastic       0.59      0.46      0.52       200

     accuracy                           0.88      1400
    macro avg       0.75      0.70      0.72      1400
 weighted avg       0.87      0.88      0.87      1400
```

```
Evaluating: roberta_tweeteval_D
Loading weights: 100% [=================================] 201/201 [00:00<00:00, 322.38it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]
  iSarcasm Macro-F1: 0.7215 (threshold=0.90)
              precision    recall  f1-score   support

not_sarcastic       0.91      0.95      0.93      1200
    sarcastic       0.60      0.45      0.51       200

     accuracy                           0.88      1400
    macro avg       0.75      0.70      0.72      1400
 weighted avg       0.87      0.88      0.87      1400


============================================================
iSarcasmEval Results Summary
           run_name emoji_mode  iSarcasm_F1  iSarcasm_threshold
roberta_tweeteval_T          T       0.7108                 0.9
roberta_tweeteval_K          K       0.7232                 0.9
roberta_tweeteval_D          D       0.7215                 0.9
```

Stage 4 Qualitative Results:

```
Total misclassified: 183 / 784
10 MISCLASSIFIED EXAMPLES

Text : Dan said he's going to have a sign for me at the airport when I fly in lol #SoCute #3MoreWeeks
True : not_irony
Predicted : irony
Prob(irony): 0.8529999852180481
Emojis : []
ESR score : 0.0

Text : @user @user A thief, a liar, a vote rigger, mocks those who use food banks and an online bully #irony
True : not_irony
Predicted : irony
Prob(irony): 0.8759999871253967
Emojis : []
ESR score : 0.0

Text : At least I don't have to clean the floor
True : not_irony
Predicted : irony
Prob(irony): 0.9139999747276306
Emojis : []
ESR score : 0.0

Text : Corny jokes are my absolute favorite
True : not_irony
Predicted : irony
Prob(irony): 0.9700000286102295
Emojis : []
ESR score : 0.0

Text : @user @user At least you got my joke
True : not_irony
Predicted : irony
Prob(irony): 0.8859999775886536
Emojis : []
ESR score : 0.0
```

```
Text : Coin ! LeCoinCanard: Coin ! LeCoinCanard: Coin ! have a wank over this hashtag: Fazzanelli: JeffRoylat have a wank over this hashtag #Not...
True : irony
Predicted : not_irony
Prob(irony): 0.1420000046491623
Emojis : []
ESR score : 0.0

Text : I just love being ignored😊|#not😒
True : irony
Predicted : not_irony
Prob(irony): 0.23600000143051147
Emojis : ['😊', '😒']
ESR score : 0.135

Text : If you wanna look like a badass, have drama on social media #not
True : irony
Predicted : not_irony
Prob(irony): 0.6259999871253967
Emojis : []
ESR score : 0.0

Text : You could say after today I'm officially necrophelic #LatinFinal #justajoke #not intodeadthings
True : irony
Predicted : not_irony
Prob(irony): 0.2630000114440918
Emojis : []
ESR score : 0.0

Text : #Hazard worker training. 1st break there was 3 doz donuts and half class outside to smoke. #fail #nomealfitters #irony @user @user
True : irony
Predicted : not_irony
Prob(irony): 0.5210000276565552
Emojis : []
ESR score : 0.0
```

References

[1] F. Barbieri, J. Camacho-Collados, L. Neves, and L. Espinosa-Anke, “TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification,” Oct. 2020, doi: https://doi.org/10.48550/arxiv.2010.12421.

[2] M. Khodak, N. Saunshi, and K. Vodrahalli, “A Large Self-Annotated Corpus for Sarcasm,” *arXiv:1704.05579 [cs]*, Mar. 2018, Available: https://arxiv.org/abs/1704.05579

[3] S. Oprea and W. Magdy, “Exploring Author Context for Detecting Intended vs Perceived Sarcasm,” *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, doi: https://doi.org/10.18653/v1/p19-1275.

[4] S. Oprea and W. Magdy, “iSarcasm: A Dataset of Intended Sarcasm,” *arXiv.org*, 2019. https://arxiv.org/abs/1911.03123 (accessed Apr. 21, 2026).

[5] P. Kralj Novak, J. Smailović, B. Sluban, and I. Mozetič, “Sentiment of emojis,” *PLOS One*, vol. 10, no. 12, Dec. 2015, doi: https://doi.org/10.1371/journal.pone.0144296.

[6] Y. Liu *et al.*, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv.org*, Jul. 26, 2019. https://arxiv.org/abs/1907.11692

[7] F. Barbieri, J. Camacho-Collados, L. Espinosa Anke, and L. Neves, “TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification,” *ACLWeb*, Nov. 01, 2020. https://aclanthology.org/2020.findings-emnlp.148/

[8] D. Q. Nguyen, T. Vu, and A. T. Nguyen, “BERTweet: A pre-trained language model for English Tweets,” *arXiv:2005.10200 [cs]*, Oct. 2020, Available: https://arxiv.org/abs/2005.10200

[9] J. M. Pérez, J. C. Giudici, and F. Luque, “pysentimiento: A Python Toolkit for Sentiment Analysis and SocialNLP tasks,” *arXiv:2106.09462 [cs]*, Jun. 2021, Available: https://arxiv.org/abs/2106.09462